

Ollama local: como zerar a fatura da API do Claude Code sem perder o essencial

Registro da pesquisa feita durante a prática "O Especialista em Modelos Locais e o Dev que Está Sangrando na Fatura da API": como rodar o Claude Code com um modelo local via Ollama, resolvendo custo e privacidade sem virar refém do hardware.

Fernando Melo · 15 de setembro de 2026

01 · O PROBLEMA

Sangrando US\$ 100+/mês na API

O cenário da prática: um dev ama o Claude Code, mas está gastando cerca de cem dólares por mês na API. Alguém sugeriu rodar tudo de graça com o Ollama, localmente — só que a desconfiança é imediata: se é grátis e roda no próprio notebook, qual é a pegadinha? A qualidade despenca? E, principalmente, isso resolve a exigência do cliente de que o código não saia da máquina?

"Cara, eu amo o Claude Code, mas tô sangrando uns cem dólares por mês na API. Me disseram que dá pra rodar ele de graça com o Ollama, local. Mas eu fico de pé atrás: se é grátis e roda no meu notebook, qual é a pegadinha?"

02 · A PEGADINHA DESMASCARADA

CLI e "cérebro" são coisas separadas

O Claude Code é composto por duas partes: a interface de terminal (CLI) — que lê arquivos, roda testes e executa comandos — e o modelo de IA que toma as decisões. Trocar para o Ollama não troca a ferramenta: troca só o cérebro. A CLI continua a mesma; o que muda é o modelo por trás, que passa de um modelo Claude na nuvem para um modelo open-source (como Qwen ou Llama) rodando no próprio notebook.

CRITÉRIO	CLAUDE CODE OFICIAL (API)	CLAUDE CODE + OLLAMA (LOCAL)
Custo	Pago por token (pode passar de US\$ 100/mês)	Totalmente grátis (só energia elétrica)
Privacidade	Dados enviados para os servidores da Anthropic	100% offline — nenhum código sai da máquina
Velocidade	Rápido, processado em supercomputadores na nuvem	Mais lento, depende do hardware do notebook
Qualidade do código	Excepcional — raciocínio avançado, poucos erros	Boa a mediana — modelos menores erram mais no contexto

Não é lixo — mas tem queda real de desempenho

Modelos locais focados em programação, como o **qwen2.5-coder** ou o **codellama**, são surpreendentemente bons no dia a dia: achar bugs, refatorar funções isoladas, gerar testes unitários. A diferença aparece na capacidade de contexto — se o projeto exigir cruzar informações de dez arquivos ao mesmo tempo para resolver um problema arquitetural complexo, o modelo local patina ou esquece detalhes onde o modelo original resolveria de primeira.

Resolvida por completo

Se o contrato do cliente exige que nenhum dado seja transmitido para terceiros, o Ollama local resolve isso de forma total: todo o tráfego acontece em uma porta interna da própria máquina (**localhost:11434**). Na prática, dá até para desligar o Wi-Fi do notebook e o Claude Code continua funcionando normalmente — prova de que nada está saindo para fora.

RAM é o fator decisivo — não o processador

Como os modelos rodam inteiramente na memória, o tamanho do modelo que dá para usar depende de quanta RAM (ou VRAM) está disponível.

16 GB de RAM — limite mínimo

Rodam modelos de 7B, como o **qwen2.5-coder:7b**. Rápido, consome 5–6 GB e dá conta de tarefas diárias, geração de funções e bugs simples.

32 GB de RAM ou Mac M-series — ideal

Rodam modelos de 14B ou 32B. O **qwen2.5-coder:14b** é o "ponto doce" entre inteligência e velocidade, mais perto do Claude original em raciocínio lógico.

8 GB de RAM: inviável para desenvolvimento com agentes locais. Ficaria limitado a modelos de 1B–3B, que erram muito código.

Sem desconfigurar o ambiente atual

Não é preciso deletar a configuração existente para testar: basta instalar o Ollama, baixar um modelo de código e apontar o Claude Code para o servidor local via variáveis de ambiente.

- 1 Instalar o Ollama e baixar o modelo recomendado para programação.
- 2 Redirecionar a CLI do Claude Code para o servidor local em vez dos servidores da Anthropic.
- 3 Rodar **claude** normalmente — a interface e os comandos permanecem idênticos.

```
# Baixa o modelo de código (troque para :14b ou :32b com 32GB de RAM)
ollama run qwen2.5-coder:7b

# Redireciona a API para o Ollama local
export ANTHROPIC_BASE_URL="http://localhost:11434/v1"
export ANTHROPIC_API_KEY="local-ollama"

# Inicializa o Claude Code no projeto
claude
```

O modelo 7B (16 GB de RAM) dá conta de quê?

Ajustar testes automatizados — sim

Mapear um bug, ler o erro no terminal e corrigir a asserção ou o payload é escopo fechado. O qwen2.5-coder:7b foi treinado extensivamente em sintaxe de testes e brilha nisso.

Implementar algo arquitetural — não

Criar um microserviço do zero ou reescrever um módulo de pagamento que impacta outros 4 arquivos exige cruzar muito contexto. Modelos menores perdem a "linha de raciocínio" nesse cenário.

A melhor estratégia é o fluxo híbrido

O ganho financeiro e a conformidade de privacidade com o cliente compensam o teste. A recomendação: usar o Ollama local no dia a dia para a maior parte das tarefas (refatorações simples, bugs pontuais, testes, documentação) e, quando surgir uma tarefa de arquitetura mais pesada, limpar as variáveis de ambiente temporariamente, gastar um ou dois dólares na API oficial para resolver o problema macro, e depois voltar para o modo offline.

O que se ganha e o que se abre mão

Ganhos

Custo zero por token; privacidade absoluta (nada sai de localhost:11434); funcionamento 100% offline, inclusive sem Wi-Fi.

Trade-offs

Abre mão do topo de raciocínio da Anthropic; fica com um teto físico de RAM/VRAM; modelos menores têm suporte mais fraco a tool calling e esquecem contexto em tarefas longas.

Salvando as variáveis e ajustando o contexto

Para não digitar os comandos toda vez, as variáveis podem ser salvas no arquivo de perfil do terminal (**.zshrc** no macOS/zsh, **.bashrc** no Linux com Bash) ou de forma permanente no Windows via PowerShell. No Linux, o Ollama roda como serviço via **systemd**, o que permite parar e iniciar o serviço para liberar RAM quando não estiver programando.

Trecho ilustrativo simplificado — o comando completo grava a variável no arquivo de perfil do shell.

```
# macOS/Linux (zsh ou bash) — adiciona ao perfil do terminal
echo 'export ANTHROPIC_BASE_URL="http://localhost:11434/v1"' >> ~/.zshrc
echo 'export ANTHROPIC_API_KEY="local-ollama"' >> ~/.zshrc
source ~/.zshrc

# Linux — gerenciar o serviço do Ollama
systemctl status ollama
sudo systemctl stop ollama
sudo systemctl start ollama
```

Para voltar ao Claude original na nuvem a qualquer momento: **unset ANTHROPIC_BASE_URL** e **unset ANTHROPIC_API_KEY** na sessão atual do terminal.

Cenários ideais para o modo local

- **Código altamente sensível** — quando o sigilo exigido pelo cliente torna a nuvem inviável.
- **MVPs, automações e demos** — validar ideias rápido sem gastar nada.
- **Estudo e aprendizado** — entender como agentes de IA funcionam por dentro, sem o cronômetro do cartão de crédito rodando.